

Diseño de Arquitectura de Software: DataForge IA

1. Introducción

El presente documento describe el diseño de la arquitectura de software para **DataForge IA**, una plataforma web de análisis predictivo para Pequeñas y Medianas Empresas (PYMES). El objetivo es establecer una estructura robusta, escalable, mantenible y segura que soporte los requisitos funcionales y no funcionales identificados en fases previas del proyecto. Este diseño arquitectónico servirá como guía para el equipo de desarrollo, asegurando la coherencia y la eficiencia en la implementación del sistema.

2. Tipo de Arquitectura

Para DataForge IA, se propone una **Arquitectura Orientada a Microservicios** con una clara separación de responsabilidades, complementada por un enfoque de **Arquitectura por Capas** dentro de cada microservicio. Esta elección se justifica por la necesidad de escalabilidad independiente de los componentes, la resiliencia, la facilidad de mantenimiento y la capacidad de integrar diversas tecnologías de forma flexible. La naturaleza de procesamiento intensivo de los modelos de IA y la gestión de grandes volúmenes de datos de múltiples clientes se benefician enormemente de este paradigma [1].

La arquitectura de microservicios permitirá:

- **Escalabilidad Independiente:** Cada servicio puede escalar horizontalmente según su demanda específica.
- **Resiliencia:** El fallo de un servicio no afecta necesariamente a todo el sistema.
- **Desarrollo y Despliegue Independiente:** Los equipos pueden trabajar en servicios de forma autónoma, facilitando la integración continua y el despliegue continuo (CI/CD).
- **Flexibilidad Tecnológica:** Permite el uso de diferentes tecnologías y lenguajes de programación para distintos servicios, optimizando el rendimiento donde sea necesario.

3. Componentes Principales

El sistema DataForge IA se descompone en los siguientes microservicios y componentes principales:

- **Gateway API (API Gateway):** Punto de entrada unificado para todas las solicitudes de los clientes. Gestiona el enrutamiento, la autenticación inicial, la autorización y el balanceo de carga.
- **Servicio de Autenticación y Usuarios:** Gestiona el registro, inicio de sesión, perfiles de usuario y la gestión de roles y permisos. Se encarga de la seguridad de acceso.
- **Servicio de Gestión de Datos:** Responsable de la carga, preprocesamiento, validación y almacenamiento de los conjuntos de datos de las PYMES. Interactúa con el almacenamiento de objetos y la base de datos de metadatos.
- **Servicio de Modelado Predictivo:** Contiene la lógica para la selección, configuración, entrenamiento y ejecución de los modelos de inteligencia artificial. Es el componente central para la generación de predicciones.
- **Servicio de Visualización y Reportes:** Genera los paneles de control interactivos, gráficos y reportes. Recibe los resultados de las predicciones y los presenta de forma comprensible al usuario.
- **Servicio de Notificaciones y Alertas:** Gestiona la configuración de alertas basadas en umbrales y el envío de notificaciones a los usuarios (ej. correo electrónico, notificaciones in-app).
- **Base de Datos de Usuarios y Metadatos:** Almacena información de usuarios, roles, configuraciones de modelos, metadatos de conjuntos de datos y configuraciones de reportes/alertas.
- **Almacenamiento de Objetos (Data Lake):** Almacena los conjuntos de datos brutos y preprocesados de las PYMES de forma escalable y segura.
- **Sistema de Colas de Mensajes:** Facilita la comunicación asíncrona entre los microservicios, especialmente para tareas de procesamiento intensivo como el entrenamiento de modelos.
- **Frontend (Aplicación Web):** La interfaz de usuario que interactúa con el Gateway API para consumir los servicios del backend.

4. Relación entre Componentes

La interacción entre los componentes se realiza principalmente a través de APIs RESTful y un sistema de colas de mensajes para operaciones asíncronas:

1. **Cliente (Frontend)** se comunica con el **Gateway API**.
2. El **Gateway API** enruta las solicitudes al **Servicio de Autenticación y Usuarios** para validar credenciales y al servicio correspondiente (ej. **Servicio de Gestión de Datos**, **Servicio de Modelado Predictivo**, etc.).

3. El **Servicio de Autenticación y Usuarios** interactúa con la **Base de Datos de Usuarios y Metadatos** para almacenar y recuperar información de autenticación y autorización.
4. El **Servicio de Gestión de Datos** recibe archivos del **Gateway API**, los procesa y los almacena en el **Almacenamiento de Objetos**. También actualiza los metadatos en la **Base de Datos de Usuarios y Metadatos**.
5. El **Servicio de Modelado Predictivo** consume datos del **Almacenamiento de Objetos** y metadatos de la **Base de Datos de Usuarios y Metadatos**. Las tareas de entrenamiento de modelos se envían al **Sistema de Colas de Mensajes** para procesamiento asíncrono.
6. El **Servicio de Visualización y Reportes** consulta los resultados de las predicciones del **Servicio de Modelado Predictivo** (o de una base de datos de resultados) y los metadatos de la **Base de Datos de Usuarios y Metadatos** para generar las visualizaciones.
7. El **Servicio de Notificaciones y Alertas** monitorea los resultados de las predicciones y, al detectar umbrales, envía mensajes al **Sistema de Colas de Mensajes** para el envío de notificaciones (ej. a través de un servicio de correo electrónico externo).

5. Tecnologías Sugeridas

Componente / Capa	Tecnologías Sugeridas	Justificación
Frontend	React.js / Vue.js, TypeScript, Tailwind CSS	Frameworks modernos para UI interactiva, tipado estático, estilos eficientes.
Gateway API	Nginx, Kong / AWS API Gateway / Google Cloud Endpoints	Gestión de tráfico, seguridad, balanceo de carga, escalabilidad.
Microservicios (Backend)	Python (Django REST Framework / Flask), Node.js (Express.js)	Python para procesamiento de datos y IA; Node.js para servicios I/O intensivos.
Base de Datos (Usuarios y Metadatos)	PostgreSQL (Relacional)	Integridad transaccional, relaciones complejas, madurez.
Base de Datos (Resultados de Predicción)	MongoDB / Cassandra (NoSQL)	Flexibilidad de esquema, escalabilidad horizontal para grandes volúmenes de resultados.
Almacenamiento de Objetos	AWS S3 / Google Cloud Storage / Azure Blob Storage	Almacenamiento escalable, duradero y de bajo costo para datos brutos y procesados.
Sistema de Colas de Mensajes	RabbitMQ / Apache Kafka / AWS SQS	Comunicación asíncrona, desacoplamiento de servicios, manejo de picos de carga.
Contenedorización	Docker	Empaquetado de microservicios, consistencia de entornos.
Orquestación de Contenedores	Kubernetes / AWS ECS / Google Kubernetes Engine	Gestión de despliegue, escalado y operación de microservicios.
Modelado de IA	Scikit-learn, TensorFlow, PyTorch	Librerías robustas para desarrollo y ejecución de

Componente / Capa	Tecnologías Sugeridas	Justificación
		modelos predictivos.
Monitoreo y Logging	Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana)	Observabilidad del sistema, detección proactiva de problemas.

6. Diagrama de Arquitectura (Explicado Textualmente)

El diagrama de arquitectura propuesto es un **Diagrama de Componentes** que ilustra la estructura de alto nivel del sistema DataForge IA y las interacciones entre sus microservicios principales. Se visualiza una capa de cliente (Frontend) que interactúa con un Gateway API. Este Gateway API es el punto de entrada a los diversos microservicios del backend: Autenticación y Usuarios, Gestión de Datos, Modelado Predictivo, Visualización y Reportes, y Notificaciones y Alertas. Cada uno de estos microservicios se comunica con bases de datos específicas (Base de Datos de Usuarios y Metadatos, Base de Datos de Resultados de Predicción) y con un Almacenamiento de Objetos centralizado para los datos brutos. Un Sistema de Colas de Mensajes actúa como un bus de comunicación asíncrona entre los microservicios, especialmente para tareas de procesamiento intensivo. La infraestructura subyacente es una plataforma de nube que proporciona los servicios de cómputo, red y almacenamiento necesarios.

Código PlantUML para el Diagrama de Componentes:

```
@startuml
```

```
skinparam componentStyle rectangle
```

```
node
```

```
"Cliente" as Client {  
    [Frontend]  
}
```

```
node "Plataforma Cloud" as Cloud {  
    component "Gateway API" as APIGateway  
    component "Servicio de Autenticación y Usuarios" as AuthSvc  
    component "Servicio de Gestión de Datos" as DataMgmtSvc  
    component "Servicio de Modelado Predictivo" as PredictionSvc  
    component "Servicio de Visualización y Reportes" as VizReportSvc  
    component "Servicio de Notificaciones y Alertas" as NotifAlertSvc  
  
    database "Base de Datos de Usuarios y Metadatos" as UserMetaDB  
    database "Base de Datos de Resultados de Predicción" as PredictionResultsDB  
    cloud "Almacenamiento de Objetos (Data Lake)" as ObjectStorage  
    queue "Sistema de Colas de Mensajes" as MessageQueue  
}
```

```
Client → APIGateway : "Solicitudes HTTP/S"
```

```
APIGateway → AuthSvc : "Autenticación/Autorización"
```

```
APIGateway → DataMgmtSvc : "Carga de Datos"
```

```
APIGateway → PredictionSvc : "Solicitudes de Predicción"
```

```
APIGateway → VizReportSvc : "Visualización/Reportes"
```

```
APIGateway → NotifAlertSvc : "Configuración de Alertas"
```

```
AuthSvc → UserMetaDB : "Lectura/Escritura"
```

```
DataMgmtSvc → ObjectStorage : "Almacenar Datos Brutos/Procesados"
```

```
DataMgmtSvc → UserMetaDB : "Almacenar Metadatos"
```

```
PredictionSvc → ObjectStorage : "Leer Datos para Modelos"
```

```
PredictionSvc → UserMetaDB : "Leer Configuración de Modelos"
```

```
PredictionSvc → MessageQueue : "Enviar Tareas de Entrenamiento"
```

```
PredictionSvc → PredictionResultsDB : "Almacenar Predicciones"
```

```
VizReportSvc → PredictionResultsDB : "Leer Predicciones"
VizReportSvc → UserMetaDB : "Leer Configuración de Reportes"

NotifAlertSvc → PredictionResultsDB : "Monitorear Predicciones"
NotifAlertSvc → MessageQueue : "Enviar Notificaciones"

MessageQueue → PredictionSvc : "Procesar Tareas de Entrenamiento"
MessageQueue → NotifAlertSvc : "Procesar Envío de Notificaciones"

@enduml
```

7. Referencias

[1] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

- Documento de Visión del Sistema: DataForge IA.
- Especificación de Requisitos de Software (SRS): DataForge IA.
- Documento de Casos de Uso: DataForge IA.
- Historias de Usuario: DataForge IA.
- Análisis de Stakeholders: DataForge IA.
- Documento de Levantamiento de Requerimientos: DataForge IA.
- Estudio de Factibilidad: DataForge IA.
- Análisis del Problema: DataForge IA.